

Software Engineering Intern Task – Blog Website with Authentication & Role-Based Access

Objective

Build a simple blog website where users can create, view, and manage posts, with role-based access control. The website should display different options based on the user's role (Admin or User).

Core Features

Authentication

- Users must be able to:
 - Register
 - Login / Logout
- Store passwords securely (hashed)

Roles

- Minimum roles: Admin and User
- Roles must be stored in the database

Blog Functionality

Role	Permissions
------	-------------

User	Create, view, edit, delete own posts
------	--------------------------------------

Admin	View all posts, delete any post, optionally manage users
-------	--

- Backend must enforce all permissions, not just frontend hiding

Frontend / UI

- Display different UI elements depending on role:
 - Admin dashboard vs User dashboard
 - Hide/show actions based on permissions
-

Technical Requirements

- Backend: Any web framework (Node.js/Express, Django, Flask, etc.)
 - Database: Relational or document-based (SQLite, PostgreSQL, MongoDB)
 - API Endpoints:
 - Auth (/register, /login)
 - CRUD operations for posts
 - Use Git for version control
-

Optional Bonus

- Comments system for posts
 - Pagination for long lists of posts
 - Deployment (Heroku, Railway, Vercel, etc.)
 - Middleware for role checks or enhanced security
-

Reporting / Documentation

- Include a README explaining:
 - Setup instructions
 - How authentication and role-based access were implemented
 - Any assumptions or design decisions
 - Screenshots or short demo video (optional)
-

Evaluation Criteria

- Correct implementation of authentication & authorization
- Proper database design (relationships between users and posts)
- Clean code structure and readability
- Frontend UI correctly reflects role-based access

- Optional: bonus features add value
-

Key Notes for Candidates:

- Focus is on secure and correct backend logic more than flashy UI
- Optional features are bonus points, not mandatory
- This task tests real-world engineering fundamentals: auth, roles, CRUD, database design, and API design